

Randomizador de horarios

Maria Paula Pérez Meléndez
Matemáticas Discretas I
Universidad Nacional de Colombia
Bogotá, Colombia

Abstract—Seleccionar horario es un problema que cualquier estudiante universitario enfrenta cada semestre, generalmente esta selección se realiza de manera manual. Este trabajo presenta un generador de horarios que modela la oferta académica como un grafo de conflictos, donde cada nodo representa un grupo de una materia y cada arista conecta dos grupos cuyos horarios se traslapan. Encontrar una combinación de horario válida equivale a hallar, en ese grafo, un conjunto de nodos sin aristas entre sí que contenga exactamente un nodo por materia. Se implementaron y compararon dos algoritmos: uno por búsqueda exhaustiva que evalúa todas las combinaciones posibles filtrándolas contra el grafo, y uno por backtracking con poda que descarta ramas inválidas de forma temprana. El sistema se validó con una base de datos real extraída del Sistema de Información Académica (SIA) de la Universidad Nacional de Colombia, correspondiente a la oferta de 7 materias en el semestre 2026-1. El backtracking generó las mismas propuestas válidas que la búsqueda exhaustiva en menor tiempo, con soporte adicional para restricciones de cupos, calificación máxima y materias con grupo fijo. El resultado principal es un generador funcional, reproducible sin credenciales externas y validado contra un horario real.

Index Terms—grafos de conflicto, conjuntos independientes, backtracking, lógica proposicional, horarios académicos

I. INTRODUCCIÓN

La construcción de un horario académico es una tarea que cada estudiante universitario repite cada semestre, generalmente de forma manual: revisar la oferta de grupos, cruzar franjas horarias y evitar conflictos entre materias. Este proceso es propenso a errores, consume tiempo y rara vez tiene en cuenta más de un criterio a la vez, por ejemplo, evitar choques y, simultáneamente, priorizar profesores mejor calificados. La generación de horarios a su vez es propensa a sufrir cambios repentinos, ya que si se acaban los cupos antes de la inscripción por parte del estudiante se debe reorganizar los posibles horarios.

Este proyecto aborda ese problema mediante un generador de horarios en Python, alimentado con datos reales extraídos del Sistema de Información Académica (SIA) de la Universidad Nacional de Colombia. Vale la pena aclarar el alcance real de esta entrega frente a la propuesta inicial: dado que se trata de un proyecto individual con un periodo de trabajo corto, el esfuerzo se concentró en el núcleo algorítmico y su fundamento matemático, la generación correcta y eficiente de propuestas de horario sin choques, validada contra un horario real, en lugar de dispersarlo entre la obtención de datos, el algoritmo, una interfaz gráfica y un análisis de datos extenso al mismo tiempo. La interfaz gráfica mencionada en la propuesta

inicial queda fuera del alcance de esta entrega y se deja como trabajo futuro (sección VIII).

Vale la pena resolver este problema porque, más allá de la utilidad práctica inmediata, permite ilustrar de forma concreta cómo varios conceptos de matemáticas discretas (grafos, relaciones, lógica proposicional) convergen naturalmente en un mismo problema de combinatoria.

II. PREGUNTA A ABORDAR

¿Cómo optimizar la selección de horarios según la cantidad de materias y la oferta académica de un semestre, evitando choques y priorizando mejores profesores?

III. FUNDAMENTOS DE MATEMÁTICAS DISCRETAS

A. Teoría de grafos

El problema de generar un horario sin choques se modela como un problema sobre grafos [3], [1]. Se define un grafo no dirigido $G = (V, E)$ donde:

- Cada nodo $v \in V$ representa un grupo específico de una materia, identificado por el par (materia, grupo).
- Existe una arista $(v_1, v_2) \in E$ si y solo si v_1 y v_2 pertenecen a materias distintas y al menos una de sus sesiones semanales se traslapa en día y hora.

Con esta definición, seleccionar una combinación de horario sin choques y eligiendo exactamente un grupo por cada materia deseada, equivale a encontrar, en G , un conjunto de nodos sin aristas entre sí (un conjunto independiente) que contenga exactamente un nodo de cada partición inducida por las materias.

Este problema está relacionado con el coloreo de grafos: si cada nodo se colorea según su materia, una asignación de horario válida corresponde a elegir, de cada clase de color, un nodo sin aristas hacia los nodos elegidos en las demás clases. Esta es una restricción más fuerte que el coloreo propio clásico, pero comparte su naturaleza combinatoria y su complejidad: en el caso general, decidir si existe tal selección es un problema de complejidad alta, lo que motiva el uso de una búsqueda con poda (backtracking) en lugar de una enumeración exhaustiva simple (sección V).

La Figura 1 ilustra el concepto de coloreo de grafos con un ejemplo reducido de 3 materias. Los nodos del mismo color pertenecen a la misma materia (es decir, forman una partición); las líneas punteadas indican pares de grupos cuyos horarios chocan. Los nodos con borde grueso son un ejemplo de conjunto independiente válido: un nodo por materia, sin ninguna arista entre ellos.

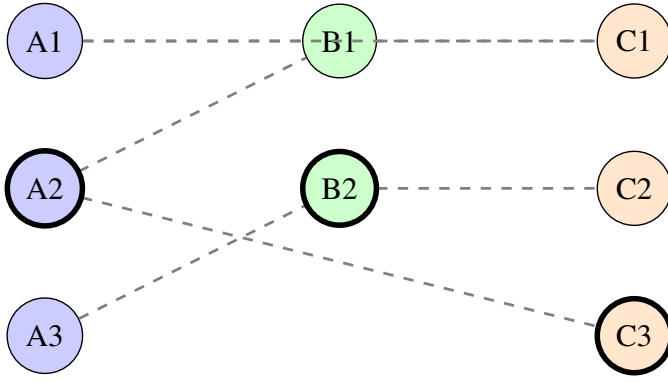


Fig. 1. Ejemplo reducido de grafo de conflicto con 3 materias (A, B, C). Las líneas punteadas son conflictos de horario. Los nodos con borde grueso (A2, B2, C3) forman un conjunto independiente válido: un grupo por materia, sin aristas entre ellos.

B. Relaciones y estructuras de conjuntos

Un horario puede definirse formalmente como un subconjunto del producto cartesiano entre los conjuntos relevantes del problema [3], [1]:

$$H \subseteq M \times S \times T \times P$$

donde M es el conjunto de materias, S el conjunto de salones, T el conjunto de franjas de horarios y P el conjunto de profesores. Cada tupla $(m, s, t, p) \in H$ representa que la materia m se dicta en el salón s , en el horario t , con el profesor p .

Esta representación permite expresar con precisión las propiedades que debe cumplir H para ser un horario válido, por ejemplo:

- **Funcionalidad respecto a (s, t) :** dado un salón y una franja horaria fijos, máximo puede existir una materia y un profesor asignados.
- **Inyectividad parcial:** para un estudiante específico, cada materia debe corresponder a una única sección y horario.

En la implementación, cada grupo puede tener varias sesiones semanales (por ejemplo, lunes y miércoles), que no necesariamente comparten franja horaria entre sí; esto se modela permitiendo que cada nodo del grafo tenga asociada una lista de sesiones, en lugar de una sola.

C. Lógica proposicional

Generar un horario válido equivale a resolver un problema booleano [3], [1]. En este caso se hace uso de lógica proposicional para formalizar las restricciones del problema y conectarlas con la implementación del código.

$x_{i,j}$ = “el estudiante toma el grupo j de la materia i ”

1) **Conflicto de horario:** Sean p y q dos de estas variables proposicionales, donde p representa que el estudiante toma un grupo j de la materia i , y q representa que toma un grupo k de una materia distinta i' (con $i \neq i'$). Si estos dos grupos se

traslapan en horario, es decir, corresponden a un par de nodos conectados por una arista en el grafo de conflictos, no pueden tomarse simultáneamente:

$$\neg(p \wedge q) \equiv (\neg p \vee \neg q)$$

La conjunción de ambas condiciones expresa que se toma *exactamente* un grupo por materia.

2) **Conexión con lo booleano:** La fórmula global del problema es la conjunción de todas las restricciones de conflicto (una por cada arista del grafo) y todas las restricciones de “exactamente un horario por materia” (una por cada materia). Una asignación de valores de verdad a las variables $x_{i,j}$ que haga verdadera esta fórmula corresponde exactamente a una propuesta de horario válida. En otras palabras, el problema de generar un horario es, en su forma más general, un problema de satisfacibilidad booleana (SAT), consiste en determinar si existe una asignación de valores de verdad para un conjunto de variables booleanas de modo que una fórmula lógica global sea verdadera [7]. Formalmente, dada una fórmula en lógica proposicional ϕ expresada generalmente en Forma Normal Conjuntiva (CNF):

$$\phi = \bigwedge_i C_i \quad (1)$$

donde cada cláusula C_i es una disyunción de literales ($C_i = l_1 \vee l_2 \vee \dots \vee l_k$), el objetivo es hallar una función de interpretación $I : V \rightarrow \{0, 1\}$ tal que ϕ se evalúe como verdadera.

Demostrado por Stephen Cook en 1971, SAT posee una importancia histórica y matemática crucial al ser el primer problema clasificado como NP-completo [7]. Esto implica que cualquier problema perteneciente a la clase NP puede reducirse en tiempo polinomial a una instancia de SAT. Por consiguiente, modelar la asignación de horarios bajo este enfoque permite justificar analíticamente la alta complejidad computacional del problema ante el crecimiento de la oferta académica, motivando el diseño de estrategias de poda combinatoria en lugar de búsquedas ciegas.

El algoritmo de backtracking implementado (sección V) puede entenderse como una búsqueda simplificada sobre esta fórmula, que aprovecha la estructura particionada del problema para podar ramas tan pronto una asignación parcial viola alguna cláusula de conflicto.

IV. DISEÑO DE LA SOLUCIÓN

Entrada: un archivo con la oferta de materias de un semestre (materia, grupo, profesor, cupos, días, hora de inicio, hora de fin, calificación del profesor), y una lista de materias a cursar, con restricciones opcionales de calificación mínima/máxima, inclusión de profesores sin calificación registrada, y materias con grupo fijo.

Salida: una lista de propuestas de horario sin choques, ordenadas por la suma o promedio de calificación de los profesores involucrados.

Restricciones consideradas:

- Sin choques de horario entre materias distintas.

- Calificación del profesor dentro de un rango aceptable (mínimo y, opcionalmente, máximo).
- Cupos disponibles mayores a cero.
- Posibilidad de fijar el grupo de una materia específica (por ejemplo, si ya está inscrita).

V. IMPLEMENTACIÓN

El sistema se implementó en Python 3.14, usando `pandas` para la carga y limpieza de datos, `networkx` para la representación explícita del grafo de conflictos, y la librería estándar para el algoritmo de búsqueda.

A. Construcción del grafo de conflictos

El grafo de conflictos se construye a partir de la oferta académica agrupando inicialmente todas las sesiones pertenecientes a un mismo grupo de una materia. Cada nodo representa un grupo completo, que puede estar compuesto por una o varias sesiones distribuidas durante la semana.

Posteriormente se comparan todos los pares de grupos pertenecientes a materias diferentes. Si al menos una sesión del primer grupo se traslapa temporalmente con una sesión del segundo grupo en el mismo día, se agrega una arista entre ambos nodos.

Una vez construido, el grafo permanece fijo durante toda la ejecución del algoritmo y es utilizado para verificar rápidamente si una combinación parcial de grupos contiene conflictos de horario.

B. Estructura de los datos de entrada

La información utilizada por el algoritmo proviene de una hoja de cálculo exportada desde el SIA. Cada fila corresponde a una sesión académica y contiene los siguientes campos:

- Materia.
- Grupo.
- Profesor.
- Día de la semana.
- Hora de inicio.
- Hora de finalización.
- Cupos disponibles.
- Calificación del profesor (los estudiantes).

Durante la carga de los datos se normalizan los formatos de hora y se eliminan registros incompletos, garantizando que el algoritmo opere sobre información consistente. esta normalización se da gracias a una secuencia de funciones en el código.

Funciones principales:

- `normalizar_hora_texto`: unifica los diferentes formatos de hora encontrados en la base de datos del SIA (HH:MM, HH:MM:SS y objetos de tipo `time`) en una representación única, evitando errores durante la comparación de horarios.
- `cargar_datos`: lee el archivo Excel de la oferta académica y normaliza los campos. Los datos reales del SIA presentan formatos de hora inconsistentes entre filas

(HH:MM:SS, HH:MM, y objetos de tipo hora), que se normalizan a un formato único antes de continuar.

- `hay_choque`: determina si dos sesiones académicas presentan un conflicto de horario. Primero verifica que ambas ocurran el mismo día y, posteriormente, compara los intervalos de tiempo para identificar traslapes. Esta función es utilizada tanto durante la construcción del grafo de conflictos como durante la generación de horarios.
- `backtracking`: función recursiva interna encargada de explorar las posibles combinaciones de grupos. En cada nivel selecciona un grupo para la materia correspondiente, verifica que no exista conflicto con los horarios previamente elegidos y continúa la búsqueda únicamente si la combinación parcial es válida. Cuando encuentra una solución completa, la almacena como una opción de horario.
- `construir_grafo_conflictos`: agrupa las sesiones por materia y grupo, y construye explícitamente el objeto de grafo, comparando cada par de grupos de materias distintas para determinar si sus sesiones se traslapan.
- `generar_opciones`: implementa la búsqueda por backtracking. Para cada materia, filtra primero los grupos candidatos según cupos, calificación y materia fija; luego recorre las materias en orden, seleccionando un grupo a la vez y comprobando choques únicamente contra los horarios ya elegidos. Si un grupo genera un choque, se descarta esa rama sin explorarla más, y se continúa con el siguiente grupo candidato.
- `mostrar_horario_tabla`: presenta cada solución encontrada en un formato tabular similar al horario mostrado en el SIA, facilitando la interpretación para el usuario.

a continuación se presenta el algoritmo que se usa para la generación de horarios mediante backtracking

Algorithm 1: Generación de horarios mediante backtracking

Input: Lista de materias y grupos disponibles

Output: Conjunto de horarios válidos

`Backtracking(i)`

if $i > |M|$ **then**

 Guardar horario;

return;

foreach grupo de la materia i **do**

if no existe conflicto **then**

 Seleccionar grupo;

`Backtracking(i + 1)`;

 Deshacer selección;

Decisión técnica relevante — búsqueda exhaustiva vs. backtracking: se implementó primero una versión que genera todas las combinaciones posibles con `itertools.product` la cual es una función del módulo `itertools` de python que genera el producto cartesiano de

2 o más iterables y las filtra contra el grafo de conflictos ya construido. Esta versión es sencilla pero no escalable, con las 7 materias de la base de datos, el número de combinaciones antes de filtrar es de aproximadamente 1.48 millones. La versión final usa backtracking. El backtracking es una forma sistemática de recorrer todas las configuraciones posibles de un espacio de búsqueda [8]. Esta función corta una rama en cuanto detecta un choque, sin necesidad de generar combinaciones completas que de antemano se sabe que son inválidas; además permite incorporar restricciones adicionales (cupos, calificación máxima, grupo fijo) de forma natural dentro de la misma búsqueda.

C. Análisis de complejidad

El análisis de complejidad temporal en notación Big O de ambos algoritmos depende de dos variables principales: el número de materias a inscribir (M) y el promedio de grupos disponibles por cada materia (G). La complejidad de la búsqueda exhaustiva es $\mathcal{O}(G^M)$ en el peor y el mejor caso, mientras que el backtracking con poda mantiene un peor caso de $\mathcal{O}(G^M)$ pero con un caso promedio y un mejor caso drásticamente inferiores, debido a la eliminación anticipada de ramas inválidas.

1) *Complejidad de la búsqueda exhaustiva:* El algoritmo de búsqueda exhaustiva genera de forma ciega el producto cartesiano de todas las combinaciones posibles de grupos antes de verificar si son válidas en el grafo de conflictos.

- **Peor, mejor y caso promedio:** $\mathcal{O}(G^M \cdot M^2)$.
- **Explicación:** el algoritmo debe evaluar obligatoriamente las G^M combinaciones totales. Para cada combinación de M materias, verificar la ausencia de conflictos en el grafo toma un tiempo proporcional a comparar todos los pares seleccionados, lo que requiere $\mathcal{O}(M^2)$ operaciones.

2) *Complejidad del backtracking con poda:* El algoritmo de backtracking construye la solución materia por materia en forma de árbol de decisión. Si al asignar un grupo a la materia actual se detecta un conflicto con las materias ya asignadas, la rama se corta inmediatamente (poda).

- **Peor caso:** $\mathcal{O}(G^M \cdot M^2)$. Ocurre cuando el grafo de conflictos no tiene aristas (no hay ningún cruce de horarios en toda la universidad). Al no haber conflictos, el algoritmo no puede podar ninguna rama y se ve obligado a explorar el árbol completo, igualando el costo de la búsqueda exhaustiva.
- **Mejor caso:** $\mathcal{O}(G \cdot M^2)$. Ocurre cuando el conflicto se detecta en los niveles más altos del árbol. Si los primeros grupos elegidos entran en conflicto de inmediato, el algoritmo descarta ramas gigantescas instantáneamente, sin llegar a explorar las combinaciones de las materias restantes.
- **Caso promedio (grafo de conflictos denso):** $\mathcal{O}(c^M)$, donde $c < G$. En un escenario real como el del SIA, la densidad de cruces de horarios provoca que la gran

mayoría de las ramas se poden en los primeros niveles. El crecimiento sigue siendo exponencial en teoría, pero la base de la potencia (c) es significativamente menor que el total de grupos (G), reduciendo el tiempo de ejecución a una fracción mínima.

TABLE I
COMPARACIÓN TEÓRICA DE ALGORITMOS.

Característica	Exhaustiva	Backtracking
Poda	No	Sí
Explora todas las combinaciones	Sí	No
Verifica conflictos	Al final	Durante la búsqueda
Complejidad (peor caso)	$\mathcal{O}(G^M)$	$\mathcal{O}(G^M)$
Complejidad promedio	Alta	Menor
Escalabilidad	Baja	Media/Alta

VI. PRUEBAS Y RESULTADOS

Para validar el funcionamiento de las implementaciones se utilizó una base de datos construida a partir de la oferta académica del Sistema de Información Académica (SIA) de la Universidad Nacional de Colombia correspondiente al semestre 2026-1. La base contiene información de siete asignaturas (Intensive english I, Ingeniería Económica, Matemáticas discretas I, Cálculo en varias variables, Fundamentos de electricidad y magnetismo, Estructuras de datos, Arquitectura de computadores), incluyendo grupos, horarios, profesores, disponibilidad de cupos y calificaciones docentes.

Las pruebas se realizaron ejecutando tanto la implementación basada en búsqueda exhaustiva como la versión basada en backtracking sobre el mismo conjunto de datos. El objetivo fue verificar la correcta detección de conflictos de horario, el cumplimiento de las restricciones implementadas y la capacidad de generar propuestas válidas bajo diferentes configuraciones.

TABLE II
CASOS DE PRUEBA REALIZADOS.

Caso de prueba	Resultado
Generación de horarios sin restricciones adicionales	Se obtuvieron propuestas válidas sin conflictos de horario.
Asignación de materias con grupo fijo	Todas las propuestas respetaron los grupos previamente establecidos.
Filtrado por calificación mínima y máxima	Se descartaron correctamente los grupos que no cumplían los criterios definidos.
Ausencia de soluciones válidas	El sistema detectó correctamente que no existían combinaciones factibles y notificó al usuario.

Las pruebas mostraron que ambas implementaciones generan el mismo conjunto de horarios válidos cuando se ejecutan sobre la misma oferta académica y bajo las mismas restricciones. Esto confirma que tanto la búsqueda exhaustiva como el algoritmo de backtracking resuelven correctamente el

problema de seleccionar un único grupo por materia evitando conflictos de horario.

Sin embargo, las diferencias aparecen en la estrategia utilizada para explorar el espacio de búsqueda. La búsqueda exhaustiva genera inicialmente todas las combinaciones posibles de grupos y posteriormente verifica cuáles satisfacen las restricciones del problema. En cambio, el algoritmo basado en backtracking incorpora dichas restricciones durante la construcción de la solución parcial, descartando inmediatamente aquellas ramas que presentan conflictos o incumplen los criterios establecidos.

TABLE III
COMPARACIÓN FUNCIONAL ENTRE LOS ALGORITMOS IMPLEMENTADOS.

Característica	Exhaustiva	Backtracking
Generación de todas las combinaciones	Sí	No
Poda durante la búsqueda	No	Sí
Detección temprana de conflictos	No	Sí
Filtrado previo por cupos	No	Sí
Filtrado por calificaciones	No	Sí
Asignación de materias con grupo fijo	No	Sí
Facilidad para incorporar nuevas restricciones	Baja	Alta

Los resultados evidencian que la principal ventaja del algoritmo de backtracking no radica únicamente en reducir la cantidad de combinaciones exploradas, sino también en la facilidad para integrar nuevas restricciones sin modificar significativamente la estructura del algoritmo. Durante el desarrollo fue posible incorporar filtros por disponibilidad de cupos, rangos de calificación y materias con grupo fijo reutilizando el mismo procedimiento recursivo.

Por otra parte, la implementación basada en búsqueda exhaustiva resultó útil como mecanismo de validación, ya que permite comprobar que las soluciones encontradas por el algoritmo con poda corresponden efectivamente al conjunto de horarios válidos. No obstante, al construir previamente todas las combinaciones posibles antes de aplicar las restricciones, su escalabilidad disminuye conforme aumenta el número de materias o de grupos disponibles.

TABLE IV
ESTADÍSTICAS DEL GRAFO DE CONFLICTOS GENERADO A PARTIR DE LA OFERTA ACADÉMICA.

Métrica	Valor
Número de nodos	65
Número de aristas	276
Densidad del grafo	0.1327
Grado promedio	8.49
Grado máximo	16
Grado mínimo	1

El grafo construido a partir de la oferta académica quedó conformado por 65 nodos y 276 aristas de conflicto. La densidad obtenida fue de 0.1327, lo que indica que aproximadamente el 13% de las posibles relaciones entre grupos representan conflictos de horario. En promedio, cada grupo

presenta 8.49 conflictos con otros grupos, mientras que el grupo con mayor número de conflictos alcanza un grado de 16. Estos resultados muestran que la oferta académica presenta una cantidad considerable de incompatibilidades horarias, justificando el uso de un algoritmo con poda para evitar explorar combinaciones que inevitablemente serán inválidas.

En conjunto, las pruebas realizadas muestran que ambas implementaciones resuelven correctamente el problema planteado. Sin embargo, la estrategia basada en backtracking ofrece una solución más flexible y escalable, al evitar explorar combinaciones que no pueden conducir a un horario válido y permitir incorporar restricciones adicionales de forma natural durante el proceso de búsqueda.

adicionalmente se presentan 2 figuras que reafirman el funcionamiento y practicidad del randomizador. la figura 2 corresponde a un horario real generado por un estudiante en 2026-1 y la figura 3 a otro generado por backtracking. se puede visualizar que los dos horarios son iguales y al mismo tiempo se confirma que el randomizador es exacto con las sugerencias de horario

UNIVERSIDAD NACIONAL DE COLOMBIA					
24/05/2026 - 30/05/2026					
Fecha	Programa	Grupo	Profesor	Hula	Frangia Horaria
LUNES	20200000 Matemáticas discretas 1	3 (Grupo 10)		SALÓN DE CLASE 401-208 (401 - Julio)	14:00 - 16:00
	100000000 Cálculo en varias variables	10 (Grupo 10)		AUDITORIO C - ANTONIO MARIA GOMEZ (403 - Guillermo Lillo Bono)	16:00 - 18:00
MARTES	202000000 Estructuras de datos	6 (Grupo 10)	Jordan Gomez Perdomo	SALA DE INFORMÁTICA 403-110 (403 - Guillermo Lillo Bono)	11:00 - 13:00
	202000000 Estructuras de computadores	3 (Grupo 10)		SALA DE INFORMÁTICA 403-101 (403 - Guillermo Lillo Bono)	11:00 - 13:00
MÉRCOLES	202000000 Matemáticas discretas 1	3 (Grupo 10)		SALÓN DE CLASE 401-201 (401 - Julio)	14:00 - 16:00
	100000000 Cálculo en varias variables	10 (Grupo 10)		AUDITORIO C - ANTONIO MARIA GOMEZ (403 - Guillermo Lillo Bono)	16:00 - 18:00
JUEVES	202000000 Cálculos de datos	6 (Grupo 10)	Jordan Gomez Perdomo	SALA DE INFORMÁTICA 403-110 (403 - Guillermo Lillo Bono)	11:00 - 13:00
	100000000 Fundamentos de electricidad y magnetismo	10 (Grupo 10)	JUAN PABLO GUEVARA MARILANDA JUAN PABLO GUEVARA MARILANDA JUAN PABLO GUEVARA MARILANDA	LABORATORIO DE ELECTRICIDAD - SF 404 - Ya Tabueta	09:00 - 11:00
VIERNES	202000000 Estructuras de computadores	3 (Grupo 10)		LABORATORIO DE MICROPROCESADORES (411 - laboratorios de Electrica y Mecanica)	11:00 - 13:00

Fig. 2. Horario real

Hora	Lun	Mar	MiÉ	Jue	Vie
07:00					
08:00					
09:00			Fundamentos de ele (10) Grupo 10	Fundamentos de ele (10) Grupo 10	Fundamentos de ele (10) Grupo 10
10:00			Fundamentos de ele (10) Grupo 10	Fundamentos de ele (10) Grupo 10	Fundamentos de ele (10) Grupo 10
11:00		Estructuras de dat (6) Grupo 6	Arquitectura de co (5) Grupo 5	Estructuras de dat (6) Grupo 6	Arquitectura de co (5) Grupo 5
12:00		Estructuras de dat (6) Grupo 6	Arquitectura de co (5) Grupo 5	Estructuras de dat (6) Grupo 6	Arquitectura de co (5) Grupo 5
13:00					
14:00	Matemáticas discre (3) (3) Grupo 3		Matemáticas discre (3) (3) Grupo 3		
15:00	Matemáticas discre (3) (3) Grupo 3		Matemáticas discre (3) (3) Grupo 3		
16:00	Cálculo en varias (10) Grupo 10		Cálculo en varias (10) Grupo 10		
17:00	Cálculo en varias (10) Grupo 10		Cálculo en varias (10) Grupo 10		

Fig. 3. Horario generado

En la sección de Anexos en el apéndice B se presentan evidencias del uso de la primera versión por búsqueda exhaustiva, no se incluyen en esta sección ya que solo fueron pruebas de una primera versión del código.

VII. DISCUSIÓN

Los resultados obtenidos muestran que el modelo basado en grafos representa adecuadamente el problema de la selección de horarios universitarios. A partir de la oferta académica del semestre 2026-1 se construyó un grafo de conflictos compuesto por 65 nodos y 276 aristas, lo que evidencia que existe un número considerable de incompatibilidades horarias entre grupos de distintas asignaturas. Esta representación permitió transformar un problema cotidiano en un problema combinatorio susceptible de resolverse mediante técnicas de búsqueda.

La comparación entre los dos algoritmos implementados mostró que ambos generan horarios válidos; sin embargo, el algoritmo basado en búsqueda exhaustiva requiere construir todas las combinaciones posibles antes de descartar aquellas que presentan conflictos. En contraste, el algoritmo de backtracking verifica las restricciones durante la construcción del horario y elimina tempranamente las ramas incompatibles, reduciendo significativamente el espacio de búsqueda explorado. Aunque ambos poseen la misma complejidad en el peor caso, en la práctica el backtracking resulta más adecuado para este tipo de problema debido a la cantidad de conflictos presentes en la oferta académica.

Otra ventaja observada fue la facilidad para incorporar restricciones adicionales dentro del algoritmo de backtracking. Funcionalidades como la asignación de grupos fijos, el filtrado por cupos disponibles y la selección por rangos de calificación de los profesores pudieron integrarse directamente durante el proceso de búsqueda, mientras que en la versión exhaustiva estas restricciones solo pueden aplicarse una vez generadas todas las combinaciones posibles, disminuyendo su eficiencia y flexibilidad.

Como validación adicional, una de las propuestas generadas por el algoritmo reproduce la estructura de un horario real obtenido del SIA, manteniendo los grupos previamente fijados y respetando todas las restricciones de conflicto. Esto demuestra que el modelo no solo produce soluciones teóricamente válidas, sino que también es capaz de generar horarios equivalentes a los utilizados en un contexto real.

Entre las principales limitaciones del trabajo se encuentra que la evaluación experimental se realizó sobre una única oferta académica y con un conjunto reducido de asignaturas. Además, aunque el programa prioriza profesores según su calificación y considera la disponibilidad de cupos, aún no incorpora preferencias personales del estudiante, como evitar jornadas muy extensas, minimizar huecos entre clases o priorizar clases ciertos días de la semana. Estas características podrían incorporarse en futuras versiones sin modificar la estructura general del modelo, gracias a la flexibilidad del algoritmo de búsqueda implementado.

VIII. CONCLUSIONES Y TRABAJO FUTURO

El objetivo principal del proyecto se cumplió al desarrollar un sistema capaz de generar automáticamente horarios académicos válidos a partir de la oferta de asignaturas del SIA, evitando conflictos entre materias y permitiendo incorporar

restricciones adicionales como grupos fijos, disponibilidad de cupos y criterios de calificación de profesores. Las pruebas realizadas con datos reales demostraron que el algoritmo encuentra correctamente combinaciones factibles y que el modelo representa adecuadamente el problema de selección de horarios.

Desde el punto de vista de las Matemáticas Discretas, el proyecto permitió aplicar de manera integrada conceptos de teoría de grafos, relaciones, conjuntos y lógica proposicional en un problema real. La representación mediante un grafo de conflictos facilitó la detección de incompatibilidades entre grupos, mientras que la formulación lógica permitió comprender el problema como un problema de satisfacibilidad booleana, justificando su naturaleza combinatoria y la necesidad de utilizar estrategias de búsqueda eficientes.

La comparación entre la búsqueda exhaustiva y el algoritmo de backtracking mostró que, aunque ambos producen los mismos horarios válidos, la poda temprana del backtracking evita explorar una gran cantidad de combinaciones imposibles, lo que lo convierte en una alternativa mucho más adecuada para situaciones con un mayor número de materias y grupos disponibles.

Como trabajo futuro, el proyecto puede extenderse incorporando una interfaz gráfica para facilitar la interacción con el usuario, ya que en las versiones actuales el uso del generador es de difícil comprensión para un usuario que no tenga conocimientos en programación. A su vez se podrían incluir nuevas preferencias de optimización (como minimizar espacios libres entre clases o evitar horarios muy tempranos o muy tarde), validaciones automáticas con conjuntos de datos extraídos automáticamente del SIA y la comparación del algoritmo implementado con técnicas de optimización más avanzadas para evaluar su comportamiento en instancias de mayor tamaño.

IX. CONTRIBUCIONES DE LA INTEGRANTE

Este es un proyecto individual por lo cual, Maria Paula Pérez Meléndez fue responsable del planteamiento del problema, la formalización matemática (grafos, conjuntos y lógica proposicional), el desarrollo completo del código (carga de datos, construcción del grafo de conflictos, y los dos algoritmos de búsqueda), la limpieza y validación de la base de datos real del SIA, y la redacción de este artículo. El repositorio fue revisado por Jesús David Correa Sierra, compañero de la asignatura, cuya revisión queda registrada en el historial de pull requests del repositorio.

APPENDIX A REPOSITORIO GITHUB

https://github.com/mapaulapm/MDI_2026_RandomizadorHorarios/tree/main

APPENDIX B
PRUEBAS VERSIÓN 1

Grafo de conflictos: 65 nodos, 276 aristas
Arquitectura de computadores (2016697): 5 grupos
Cálculo en varias variables (1000006-B): 15 grupos
Estructuras de datos (2016699): 6 grupos
Fundamentos de electricidad y magnetismo (1000017-B): 20 grupos
Ingeniería económica (2015703): 11 grupos
Inglés Intensivo I (Intensive English I) (2026054): 5 grupos
Matemáticas discretas I (2025963): 3 grupos

Fig. 4. estadísticas

20 propuestas válidas encontradas
Propuesta 4 | Calificación promedio: 3.79
Arquitectura de computadores (2016697) | (2) Grupo 2 | Francisco Meliá Orozco | Calif: 3 | Martes 11:00-11:00; Jueves 11:00-11:00
Cálculo en varias variables (1000006-B) | (12) Grupo 12 | Lucía Sánchez | Calif: 3 | Lunes 11:00-11:00; Miércoles 11:00-11:00
Estructuras de datos (2016699) | (2) Grupo 1 | Juan Alexander López-Rojas | Calif: 4 | Martes 16:00-16:00; Jueves 16:00-16:00
Fundamentos de electricidad y magnetismo (1000017-B) | (12) Grupo 12 | Frank Nubillo Rosales Rosales | Calif: 4 | Jueves 16:00-16:00; Jueves 9:00-11:00; Lunes 9:00-11:00
Ingeniería económica (2015703) | (2) Grupo 1 | Juan Sebastián Gómez García | Calif: 4 | Lunes 16:00-16:00; Miércoles 16:00-16:00
Inglés Intensivo I (Intensive English I) (2026054) | (1) Grupo 1 | Martín Felipe Suárez García | Calif: 4 | Martes 7:00-9:00; Miércoles 7:00-9:00; Viernes 7:00-9:00
Matemáticas discretas I (2025963) | (3) Grupo 3 | Juan Sebastián López García | Calif: 4 | Lunes 16:00-16:00; Miércoles 16:00-16:00

Fig. 5. Horario generado en la versión 1 del código

APPENDIX C
OTRAS PRUEBAS BACKTRACKING

Grafo de conflictos: 65 nodos, 276 aristas
No se encontraron opciones válidas con los filtros aplicados.

Fig. 6. Horario con calificación mínima de 8

OPCIÓN 1 Balance de Calificación Profesores = 18						
Hora	Lun	Mar	MiÉ	Jue	Vie	
07:00						
08:00						
09:00	Fundamentos de ele (12) Grupo 12			Fundamentos de ele (12) Grupo 12		
10:00	Fundamentos de ele (12) Grupo 12			Fundamentos de ele (12) Grupo 12		
11:00	Cálculo en varias (12) Grupo 12	Arquitectura de co (2) Grupo 2	Cálculo en varias (12) Grupo 12	Arquitectura de co (2) Grupo 2		
12:00	Cálculo en varias (12) Grupo 12	Arquitectura de co (2) Grupo 2	Cálculo en varias (12) Grupo 12	Arquitectura de co (2) Grupo 2		
13:00						
14:00	Matemáticas discre (3) Grupo 3		Matemáticas discre (3) Grupo 3	Fundamentos de ele (12) Grupo 12		
15:00	Matemáticas discre (3) Grupo 3		Matemáticas discre (3) Grupo 3	Fundamentos de ele (12) Grupo 12		
16:00		Estructuras de dat (3) Grupo 3		Estructuras de dat (3) Grupo 3		
17:00		Estructuras de dat (3) Grupo 3		Estructuras de dat (3) Grupo 3		
18:00						

Fig. 7. Horario generado sin restricciones

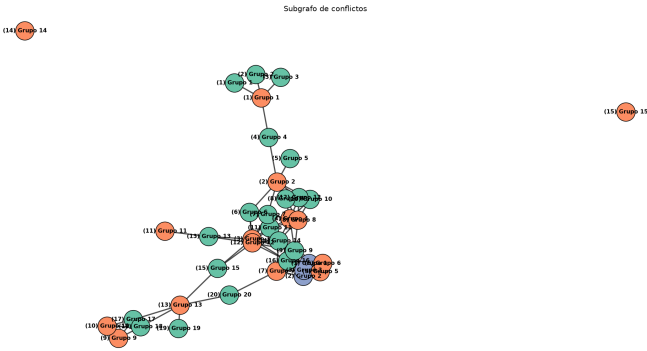


Fig. 8. Grafos de 3 materias de la base de datos

REFERENCES

- [1] J. F. Villalpando Becerra and A. García Sandoval, *Matemáticas discretas, aplicaciones y ejercicios*. Ciudad de México, México: Grupo Editorial Patria, 2014.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [3] L. Lovász, J. Pelikán, and K. Vesztergombi, *Discrete Mathematics: Elementary and Beyond*. New York, NY, USA: Springer, 2003.
- [4] NetworkX Developers, *NetworkX Documentation*, 2024. [Online]. Available: <https://networkx.org/documentation/stable/>
- [5] The pandas development team, *pandas Documentation*, 2024. [Online]. Available: <https://pandas.pydata.org/docs/>
- [6] Universidad Nacional de Colombia, *Sistema de Información Académica (SIA)*. [Online]. Available: <https://sia.unal.edu.co>
- [7] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158. doi: 10.1145/800157.805047.
- [8] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [9] P. Fernández Gallardo and J. L. Fernández Pérez, “Coloreado de grafos,” in *El discreto encanto de la matemática*, ch. 10.2, Universidad Autónoma de Madrid, 2016, pp. 1–31. [Online]. Available: <https://verso.mat.uam.es/~pablo.fernandez/entrega6-EDEM-MD16-17.pdf>